

14.04.2024г.

Лекция 7 и 8

Тема 7A / 7B

Complex

Nvector

Предефиниране на оператори

Operation Overloading

Оператор-функция, но със специален синтаксис

Имаме три вида оператори:

- унарни $+, -, ++, --$
- бинарни $+, -, /, *$
- тернарни $?:$

Операторите се характеризират с три неща:

- приоритет $a \& b @ c$
- асоциативност - лева $(())$ $(a+b)+c+d$
- десна $(())$

$f() + g()$ // не знаем коя ще се изпълни първа
защото не са предефинирани

- позиция на оператора спрямо аргумента

- инфиксни
- суфиксни
- префиксни

```
struct A { ... }
```

```
  A obj1;
```

```
  A obj2;
```

```
  obj1 + obj2;
```

по 2 начина стават предефинирани

• външна функция

```
A operator $(const A&, const A&)
```

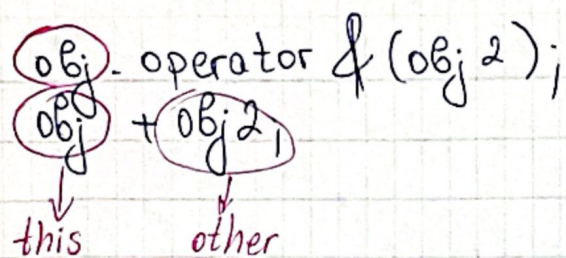
```
operator $(obj1, obj2);
```

```
obj1 & obj2;
```

• предефиниране като вътрешна функция

```
A { A operator $(const A& obj)
```

```
  A obj1, obj2;
```

Еквивалентни са
двата записа

⚠ Левият обект е този, от който се извлича оператори.

⚠ Уговорка:
 $+=$, $-=$, $*=$, $/=$ да бъдат вътрешни функции
 $+$, $-$, $*$, $/$, $==$, за сравнение да са външни

При предефиниране на оператор, трябва да се спазват общо приети стандарти.

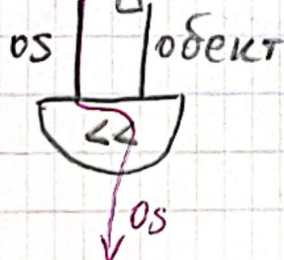
Пример 1: $+$ не променя аргументите си и
 връща нов обект
 $+=$ променя левия аргумент и
 връща референция към левия обект
 $==$ и оператори за сравнение
 връщат bool

Особени случаи:

$<<$ и $>>$

ostream << обект

→ задължително са външни функции,
 защото потока трябва да е от ляво



// тези оператори са
 ляво-асоциативни

- Предефиниране на оператори за инкрементация и $\&$

$a++$ - $(a-1)$ - връща старата стойност на a

$++a$ - ще се замени a (нова стойност)

↓ $A \& \text{operator } ++()$?

...
return *this;

$A \text{ operator } ++(\text{int})$

this → x++;

return $A(x-1)$;

↓ dummy parameter (ще не се използва) // така правим разликата между суфикс и префикс

връща нов обект със старата стойност



Некои оператори трябва да са елен-функции
=, [], (), →

↓
да разгледам този оператор за теория и изпит



Ограничения

- Некои оператори не могат да се предефинират
пример: $::?$ $:::$ $:::$
- Нови оператори не могат да се създават.
- Не може да предефинираме асоциативност, приоритет, позицията спрямо аргумент

- оператор → - трябва да връща обект (не може да е void) (указател / референция / копие)
- Ако предефинираме $\&$ и $||$ губим предварителното изчисление.

Приятелски класове и функции

Класове / функции, които имат достъп до private член-ранните.

```
struct A: {  
    private:  
        int x;  
    public:
```

```
    friend void f();
```

→ външна функция

```
    friend class B; // в класа B можем да  
    }               използваме x
```

```
void f()  
{ A obj;  
  obj.x++;  
}
```

```
class B {  
    A obj;  
    B() { obj.x = 7; }  
}
```

```
class X  
{
```

```
    friend class Y  
}
```

```
class X Y  
{
```

```
    friend class Z  
}
```

Z не е "friend" на X

Complex → статична памет
+, +=, ..., *=, /=, <<, >>

Nvector → динамична памет
2x[]

Δ [] се разписва 2 пъти.
върху константа / неконстанта

Дефинираме $\oplus$ и го използваме за $\oplus$, но не винаги е така

(контра пример):

при стринг

// не използваме директно $\oplus$

// задължително трябва да мога да пиша стринг

Тема 8

Статик

1) Статична функция

A.cpp

`void f() { ... }`

1) може да бъде използвана от други .cpp файлове

2) друга функция `f()` не може да дефинира в друг файл

→ `static void f() { ... }`

→ `namespace { f(); }`

Така функцията `f()` се ограничава до 1 компилационна единица (т.е. 1 .cpp файл)

2) Статична променлива в тялото на функцията

`f() {`

`A obj;`

`static B obj2;`

`C obj3;`

→ общ за всички извиквания на функцията

`int main() {`

`f(); A() B() C(), ~C, ~A` // не викаме `~B`, защото е готова за следващото извикване на `f()`

`f(); A() C() ~C ~A` // вече

`~B()` // не е сигурно `B` е създаден и го използваме

Създаване на обекта

- при първото извикване на функцията

Изтриване

- при излизане от програмата

3) Статични член-променливи

```
class X {
```

```
    A obj;
```

```
    static B obj2; // нещо като глобален обект
```

```
}
```

Не е обвързан с конкретна инстанция на
глобален обект, ~~който е~~ енкапсулиран
в клас X

X.cpp

```
B x::obj2;
```

деф. конструктор на B

само върне



B не влияе на големината на X.

За всички създадени обекти на X, ще има едно и също B.

```
class Y {
```

играе роля на на
патерна за obj2.

```
public:
```

```
    static B obj2;
```

```
}
```

```
main() {
```

```
    X obj;
```

```
    Y::obj2
```

```
    obj.obj2;
```

// еквивалентни са, но

// не го използваме

4) Статична член-функция

```
class Z {
```

```
    int x;
```

```
    static int y;
```

```
public:
```

```
    f()
```

```
    static f();
```

~ достъпване

// не ни трябва обект, за

внешно функция // да я извикаме

Не могат да са константни / виртуални.

Статичните функции имат достъп само до статичните член-данни на класа.

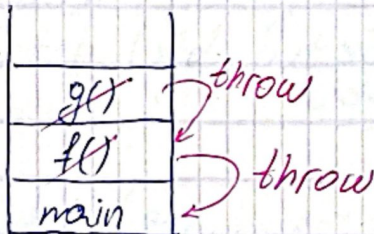
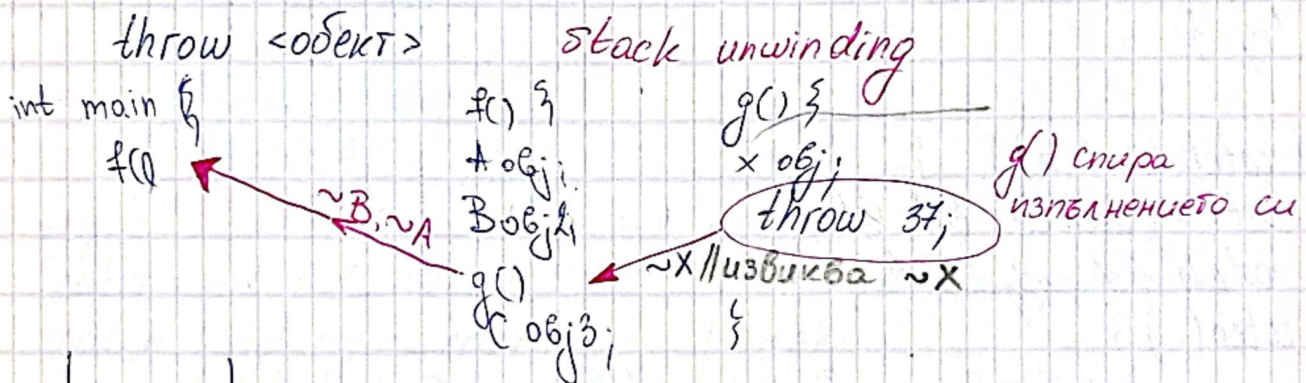
Статичен клас:

Клас, който има само статични член-данни.

Изключения

Exceptions

def. → Сигнал, че има проблем.



```
try {  
    // проблем  
    код
```

```
} catch (int x)  
{  
    // обработка на грешката  
}
```


main

```

try {
    obj;
    f();
} catch (int x) {
    cout << "Error" << x;
}

```

f() {
...
g();
...
}

Stack unwinding става само, ако някой обработи грешката.
 ↳ Ако не е обработена, програмата спира `std::terminate`

try {

```

} catch (int a) { }
↓ catch (double b) { }
catch (* obj) { }
catch(...) { }
}

```

← хваща всичко // всякакви изключения

try {

```

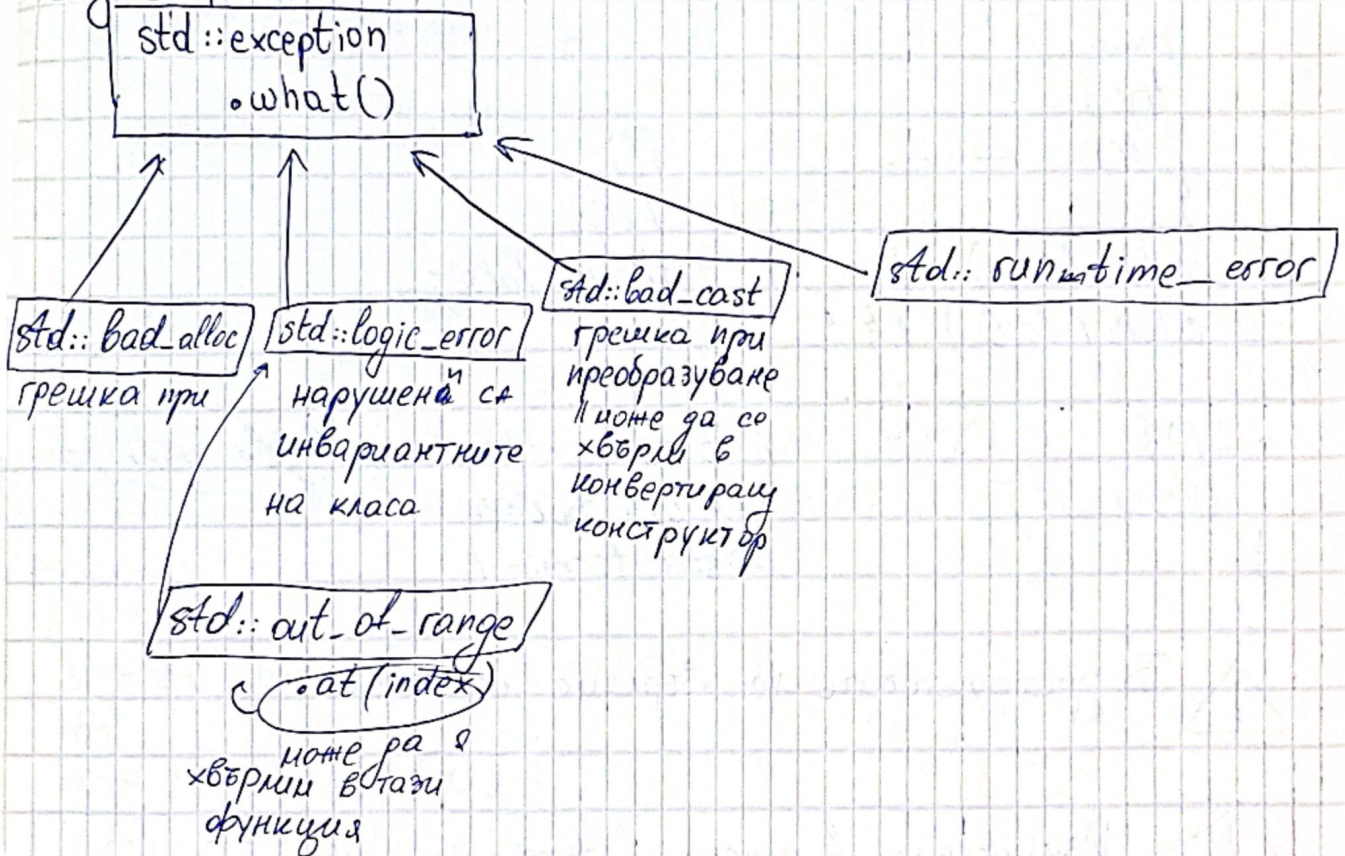
} catch (A obj) { } catch (const A& obj) { }

```

→ копие/move → следващия път

assert — наши грешки (на програмиста) / системни грешки
 exception — външни грешки

Бүгдөө грешки:



```

try {
    catch (std::bad_alloc)
    catch (std::logic_error)
    catch (std::out_of_range)
    catch :
  
```

от най-конкретната
към
най-общата грешка

```

catch (std::exception)
catch (...)
  
```



При throw std::exc("ABC")

↑
поставяме стринг

```

catch (const std::exc& e)
    e.what() // "ABC"
  
```


• Изключения при деструктори

```

main
try {
    f()
}
catch (int) {}
catch (bool) {}

f()
A obj;
throw false;
    
```

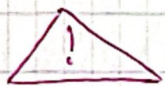
Diagram: A red arrow points from the `f()` call in the `try` block to the `f()` function definition. Another red arrow points from the `throw false;` statement to the `catch (bool) {}` block. A red line with $\sim A$ above it connects the `f()` function to the `catch (bool) {}` block.

```

~A() {
    throw 37;
}
    
```

// В този случай има две изключения или повече
 $\Rightarrow$ `std::terminate`

⚠ В деструкторите не хвърляме изключения



Изключения в конструкторите

```

A obj; A()
{
    :
    throw 37;
}
    
```

деструктора на A не се извиква

// това може да провери до опитане на памет

→ При хвърляне на изключение в конструктор се извикват деструкторите на всички напълно построени обекти в класа (приключил или е конструкция = построен)

```

X {
    A obj1;
    B obj2;
    C obj3;
}

B() { throw 37; }
    
```

X obj; // A(), B(), ~A()
 извикани от конструктор на X



При изключения в конструктор не трябва да остават незатворени външни ресурси.

```
A {  
  char* str;  
  A() {  
    str = new char[10];  
    ...  
    throw 37;  
  }
```

// няма да извика ~A()
няма кой да изтрие str

решение:

```
A()  
{  
  str = new char[10];  
  try  
  {  
    str2 = new char[70];  
  }  
  catch (const std::bad_alloc&)  
  {  
    delete[] str;  
    throw;  
  }
```

→ хвърли същата грешка